Stage 1

User Story: User Management System

As a user, I want to register, login, logout, and reset my password through a console-based Core Java application so that my user information can be managed using MySQL and JDBC.

1. User Registration

The system should allow a user to register with the following attributes:

- First Name (fname)
- Last Name (lname)
- Mobile Number (mobile)
- Email Address (email)
- Location (location)
- Last Modified Date (lastmodifieddate)
- Status (status)
- Username (username)
- Password (password)

Validations

- fname, lname, mobile, email, username, and password should be mandatory fields.
- Email format should be valid.
- Password should be at least 8 characters long with one uppercase, one lowercase, one special character, and one digit.
- Username must be unique.
- Mobile number must be valid (10 digits).

Functionality

- User should be able to register if all validations are met.
- Status should be set to Active by default upon registration.
- lastmodifieddate should be the current date/time when the user registers.
- All required inputs should be accepted using the **Scanner class**.
- User registration details should be stored in the **MySQL database using JDBC**.

2. User Login

The system should allow a user to login using:

- Email and Password OR
- Username and Password.

Validations

- Fields must not be empty.
- If the user does not exist, display an appropriate error message.
- If the login is successful, display a welcome message using the user's first and last name.

Example:

Welcome Amruta Patil

All required inputs should be accepted using the **Scanner class**.

3. Post-Login Menu

After successful login, display a console-based menu with the logged-in user's welcome message:

Welcome, fname lname

The user should have an option to **Logout**.

On selecting Logout, the user should be logged out and returned to the appropriate console menu.

4. Forget Password

The user should be able to reset their password using:

- Email OR
- Username.

Process

- The system should identify the registered user using the provided email or username.
- email or username must be validated before proceeding a request for password reset.
- The new password should follow the same password policy as registration.

User Story: User Account Lockout

As a user, I want the system to track my failed login attempts and automatically lock my account after 3 consecutive failed login attempts so that my account is protected from repeated unsuccessful login attempts.

1. Failed Login Attempts

The system should:

- Track the number of failed login attempts for each user.
- Increment the failed_attempts count after every unsuccessful login attempt.
- After 3 consecutive failed login attempts, the user account should be locked.

2. Account Locking

- Once the account is locked, the user's status in the MySQL database should be updated from active to inactive.
- A locked user should no longer be able to log in.
- The account can only be unlocked manually by an administrator.

3. Database

The system should:

- Store the user's failed_attempts in the MySQL database.
- The status field should reflect active or inactive based on the account state.
- All database operations should be performed using **JDBC**.

4. Feedback to User

The system should:

- Inform the user after each failed login attempt how many attempts remain before their account is locked.
- When the account is locked, display a message informing the user that their account is locked and they need to contact support.
- If the user successfully logs in within the given chances, the failed_attempts field should be reset to **ZERO (0)**.

All required inputs should be accepted using the **Scanner class**.